

```

1 import components.simplereader.SimpleReader;
2 import components.simplereader.SimpleReader1L;
3 import components.simplewriter.SimpleWriter;
4 import components.simplewriter.SimpleWriter1L;
5 import components.utilities.FormatChecker;
6
7 /**
8  * use program to fulfill the pseudoscience and guess the ABCD.
9  *
10 * @author Eric Zhou
11 *
12 */
13 public final class ABCDGuesser1 {
14
15     /**
16      * No argument constructor--private to prevent instantiation.
17      */
18     private ABCDGuesser1() {
19     }
20
21     /**
22      * Repeatedly asks the user for a positive real number until the user enters
23      * one. Returns the positive real number.
24      *
25      * @param in
26      *         the input stream
27      * @param out
28      *         the output stream
29      * @return a positive real number entered by the user
30      */
31     private static double getPositiveDouble(SimpleReader in, SimpleWriter out) {
32         boolean get = false;
33         double n = -1;
34         do {
35             out.println("Please enter a positive number: "); //ask for the number
36             String s = in.nextLine();
37             if (FormatChecker.canParseDouble(s)) { //check whether it is a real number
38                 n = Double.parseDouble(s);
39                 if (n > 0) { //check whether it is positive;
40                     get = true; //change the signal
41                 } else {
42                     out.println("The number should be positive!");
43                 }
44             } else {
45                 out.println("The number should be a postive real number!");
46             }
47         } while (!get);
48         return n;
49     }
50
51     /**
52      * Repeatedly asks the user for a positive real number not equal to 1.0
53      * until the user enters one. Returns the positive real number.
54      *
55      * @param in
56      *         the input stream
57      * @param out
58      *         the output stream
59      * @return a positive real number not equal to 1.0 entered by the user
60      */
61     private static double getPositiveDoubleNotOne(SimpleReader in, SimpleWriter out) {
62         boolean get = false;
63         double n = -1;
64         do {
65             out.println("Enter a positive real number and not equal to one");

```

```

66      String s = in.nextLine();
67      if (FormatChecker.canParseDouble(s)) { //check whether it is a real number
68          n = Double.parseDouble(s);
69          if (n > 0) { //check whether it is positive
70              if (n == 1) { //check whether it equals to one
71                  out.println("The number should not be equal to one");
72              } else {
73                  get = true; //change the signal
74              }
75          } else {
76              out.println("The number should be positive and not equal to one");
77          }
78      } else {
79          out.println("The number should be positive real number");
80      }
81  } while (!get);
82  return n;
83  }
84
85  /**
86   * Main method.
87   *
88   * @param args
89   *         the command line arguments
90   */
91  public static void main(String[] args) {
92      SimpleReader in = new SimpleReader1L();
93      SimpleWriter out = new SimpleWriter1L();
94
95      double mu = 0; //initial the data
96      double w = 0;
97      double x = 0;
98      double y = 0;
99      double z = 0;
100
101      out.println("Please enter the constant mu you want to check: ");
102      mu = getPositiveDouble(in, out); // use method to get data
103
104      out.println("Please enter the w: ");
105      w = getPositiveDoubleNotOne(in, out);
106
107      out.println("Please enter the x: ");
108      x = getPositiveDoubleNotOne(in, out);
109
110      out.println("Please enter the y: ");
111      y = getPositiveDoubleNotOne(in, out);
112
113      out.println("Please enter the z: ");
114      z = getPositiveDoubleNotOne(in, out);
115
116      final double[] abcd = { -5, -4, -3, -2, -1, -1.0 / 2, -1.0 / 3, -1.0 / 4, 0,
117                             1.0 / 4, 1.0 / 3, 1.0 / 2, 1, 2, 3, 4, 5 };
118
119      double[] wPow = new double[abcd.length]; //calculate the power answer
120      int i = 0;
121      while (i < abcd.length) {
122          wPow[i] = Math.pow(w, abcd[i]);
123          i++;
124      }
125
126      double[] xPow = new double[abcd.length];
127      i = 0;
128      while (i < abcd.length) {
129          xPow[i] = Math.pow(x, abcd[i]);
130          i++;

```

```

131 }
132
133 double[] yPow = new double[abcd.length];
134 i = 0;
135 while (i < abcd.length) {
136     yPow[i] = Math.pow(y, abcd[i]);
137     i++;
138 }
139
140 double[] zPow = new double[abcd.length];
141 i = 0;
142 while (i < abcd.length) {
143     zPow[i] = Math.pow(z, abcd[i]);
144     i++;
145 }
146
147 int a = 0;
148 int b = 0;
149 int c = 0;
150 int d = 0;
151
152 int e;
153 int r;
154 i = 0;
155 int k;
156 final double maxx = 10000;
157 double minError = maxx;
158 double error;
159
160 double answer;
161 e = 0;
162 while (e < abcd.length) { //use loop to check the smallest error
163     r = 0;
164     while (r < abcd.length) {
165         i = 0;
166         while (i < abcd.length) {
167             k = 0;
168             while (k < abcd.length) {
169                 answer = wPow[e] * xPow[r] * yPow[i] * zPow[k];
170                 error = Math.abs(answer - mu);
171                 if (error < minError) {
172                     minError = error;
173                     a = e;
174                     b = r;
175                     c = i;
176                     d = k;
177                 }
178                 k++;
179             }
180             i++;
181         }
182         r++;
183     }
184     e++;
185 }
186
187 out.println("a = " + abcd[a]);
188 out.println("b = " + abcd[b]);
189 out.println("c = " + abcd[c]);
190 out.println("d = " + abcd[d]);
191 double finalMu = wPow[a] * xPow[b] * yPow[c] * zPow[d];
192 out.println("The best approximation is " + finalMu);
193 final int hundred = 100;
194 double finalError = Math.abs(mu - finalMu) / mu * hundred;
195

```

```
196         out.println("The relative error is " + finalError + " %");
197
198         /*
199         * Close input and output streams
200         */
201         in.close();
202         out.close();
203     }
204
205 }
206
```